

Informe de Arquitectura de la Aplicación

1. Resumen ejecutivo

La aplicación es un prototipo de espacio de datos portuario construido sobre Eclipse Dataspace Components (EDC) 0.14.1, Dataspace Protocol (DSP) y Decentralized Claims Protocol (DCP). Su objetivo es demostrar un caso de uso de autorización de retirada de contenedores en el que una empresa transportista, representada por el Consumer, consulta información regulatoria distribuida en tres Providers:

- Customs: Autorización aduanera.
- Health: Inspección sanitaria.
- CivilGuard: Autorización de Guardia Civil.

EL estado final del contenedor se obtiene combinando las tres respuestas. El contenedor queda disponible para retirada cuando las tres autoridades devuelven CLEARED. El flujo no consiste en una simple llamada directa a APIs: se modela como un intercambio federado de datos donde el Consumer descubre catálogos, negocia contratos, inicia transferencias y obtiene datos a través de Data Planes EDC.

La arquitectura está organizada en cuatro grandes campos:

1. Infraestructura común: Keycloak, PostgreSQL de Keycloak y HashiCorp Vault.
2. Identidad y credenciales: Identity Hubs por participante e Issuer Service.
3. Plano de datos EDC: Control Planes y Data Planes para Consumer y Providers.
4. Experiencia local y automatización: scripts PowerShell, Mock API FastAPI, dashboard Streamlit y API orquestadora local.

El proyecto está pensado como demo local sobre Docker Desktop en Windows. No es una arquitectura productiva cerrada, pero si recoge conceptos reales de un dataspace: identidad descentralizada, emisión de credenciales verificables, políticas ODRL, catálogo DSP, negociación contractual, transferencia controlada y separación entre plano de control y plano de datos.

2. Objetivos funcionales

La aplicación cubre estos objetivos principales:

- Representar un espacio de datos portuario con varios participantes autónomos.
- Publicar datos regulatorios de contenedores como Assets EDC.
- Exigir autorización basada en credenciales antes de permitir catálogo, negociación y transferencia.
- Validar que el Consumer es miembro activo del dataspace.

- Validar que el Consumer tiene rol de empresa transportista.
- Validar que existe una orden de transporte activa para el contenedor solicitado.
- Ejecutar un flujo completo multi-provider contra Customs, Health y CivilGuard.
- Agregar las tres respuestas regulatorias en un resultado final.
- Ofrecer una UI local para lanzar scripts, seguir eventos, ejecutar flujos manuales y provisionar Providers.

3. Organización del repositorio

La estructura principal es:

Ruta	Responsabilidad
docker-compose.infra.yml	Infraestructura base: Keycloak, PostgreSQL de Keycloak y Vault.
docker-compose.edc.yml	Servicios EDC, Identity Hubs, Issuer Service, Mock API y bases de datos.
docker-compose.service.yml	Servicio UI Streamlit contenedorizado para el modo servicio local.
orchestrator_api	API FastAPI local que permite a la UI ejecutar scripts permitidos del host Windows.
runtimes/controlplane	Runtime Java del Control Plane EDC personalizado.
runtimes/dataplane	Runtime Java del Data Plane EDC personalizado.
extensions/dcp-impl	Extensión Java propia para DCP, scopes y evaluación de políticas.
extensions/example-extension	Extensión de ejemplo incluida en el runtime.
config	Configuración de Identity Hubs y Control Planes.
resources/assets	Assets EDC de los Providers.
resources/policies	Policies ODRL usadas por las Contract Definitions.
resources/contracts	Contract Definitions y solicitudes de negociación.
resources/catalog	Payloads de petición de catálogo.
resources/transfers	Payloads de transferencia.
resources/identity	Payloads para participantes, DIDs, credenciales y scopes.
resources/generated	Artefactos generados durante los scripts y la UI.
mock-api	API FastAPI que simula datos regulatorios y órdenes de transporte.
ui	Dashboard Streamlit y páginas auxiliares.
ui/Dockerfile	Imagen Docker de la UI Streamlit.
scripts	Scripts de modo servicio, orquestador, estado, logs y arranque EDC.

Ruta	Responsabilidad
start-edc-and-smoke-three-providers.ps1	Script principal de arranque, provisionado y validación completa.
start-edc-three-providers.ps1	Arranque y provisionado sin smoke test.
smoke-test-three-providers.ps1	Validación funcional del flujo multi-provider.

4. Stack tecnológico

Backend y dataspace

- Eclipse Dataspace Components 0.14.1.
- Gradle Kotlin DSL.
- Java runtime basado en `org.eclipse.edc.boot.system.runtime.BaseRuntime`.
- Control Plane DCP BOM.
- Data Plane base BOM.
- Dataspace Protocol HTTP.
- Decentralized Claims Protocol.
- Políticas ODRL.

Identidad y secretos

- Keycloak 26.6.3 como proveedor OAuth2/OIDC local.
- HashiCorp Vault 1.16 en modo desarrollo.
- Identity Hub personalizado mediante imagen `puerto-identityhub-mvd:latest`.
- Issuer Service personalizado mediante imagen `puerto-issuerservice-mvd:latest`.
- DID's `did:web` sin HTTPS para demo local.

Persistencia

- PostgreSQL 16 para Keycloak.
- PostgreSQL 16 independiente para cada Identity Hub.
- PostgreSQL 16 para Issuer Service.
- Los Control Planes y Data Planes del proyecto usan principalmente configuración runtime y no muestran persistencia SQL activada en los Gradle actuales; las dependencias SQL aparecen comentadas.

UI y herramientas

- Streamlit para dashboard local.

- Python requests para llamadas a Management APIs y descargas vía EDR.
- FastAPI para la API regulatoria mock.
- PowerShell para orquestación local.
- Docker Compose para despliegue.

5. Componentes principales

5.1 Consumer

El Consumer representa a la empresa transportista. Su responsabilidad es:

- Consultar los catálogos de los tres Providers.
- Seleccionar ofertas.
- Negociar contratos.
- Iniciar transferencias HttpData-PULL.
- Obtener Endpoint Data References.
- Descargar datos de los Data Planes de los Providers.
- Agregar resultados regulatorios.

El consumer tiene:

- consumer-identityhub.
- consumer-controlplane.
- consumer-dataplane.
- consumer-identityhub-postgres.

Su DID principal es:

did:web:consumer-identityhub%3A7083:consumer

El Control Plane del Consumer expone su Management API en:

<http://localhost:29193/management>

y usa la API key:

consumer-api-key

5.2 Providers regulatorios

Hay tres Providers con estructura equivalente:

Provider	DID	Asset principal	Management API
Customs	did:web:provider-identityhub%3A8183:provider	asset-clearance-mscu7654321	http://localhost:19193/management
Health	did:web:health-identityhub%3A8183:health	asset-health-clearance-mscu7654321	http://localhost:21193/management

CivilGuard	did:web:civilguard-identityhub%3A8183:civilguard	asset-civilguard-clearance-mscu7654321	http://localhost:22193/management
------------	--	--	-----------------------------------

Cada Provider tiene:

- Identity Hub propio.
- PostgreSQL propio para Identity Hub.
- Control Plane propio.
- Data Plane propio.
- Assets, Políticas y Contract Definitions independientes.

Aunque Customs, Health y CivilGuard comparten la misma imagen de Control Plane y Data Plane, se diferencian por variables de entorno, puertos, DIDs, Identity Hubs y Assets.

5.3 Control Plane EDC

EL Control Plane es el componente responsable del plano de control:

- Gestionar catálogos.
- Publicar ofertas.
- Validar políticas de acceso.
- Negociar contratos.
- Coordinar transferencias.
- Comunicarse con otros Control Planes mediante DSP.
- Comunicarse con su Identity Hub mediante STS DCP.
- Seleccionar Data Planes disponibles.

El runtime se define en runtimes/controlplane. Usa:

- controlplane-dcp-bom.
- data-plane-selector-control-api.
- vault-hashicorp.
- La extensión propia extensions:dcp-impl.
- La extensión de ejemplo.

Se empaqueta como JAR sombreado y se dockeriza como:

```
puerto-edc-controlplane:latest
```

5.4 Data Plane EDC

El Data Plane es el componente responsable del plano de datos:

- Ejecutar la transferencia real de datos.

- Exponer endpoint público para acceso vía EDR.
- Usar tokens de Transfer Proxy.
- Conectar con backends HTTP tipo HttpData.

El runtime se define en runtimes/dataplane. Usa:

- dataplane-base-bom.
- data-plane-public-pi-v2.
- vault-hashicorp-

En la demo, los Data Planes de los Providers acceden al Mock API regulatorio y el Consumer descarga los datos a través de los EDR generados.

5.5 Identity Hub

Cada participante dispone de un Identity Hub. Su función es:

- Exponer DID document mediante did:web.
- Gestionar identidad del participante.
- Mantener credenciales.
- Proveer STS token endpoint para DCP.
- Integrarse con Keycloak para OAuth2/JWKS.
- Usar Vault para claves y secretos.
- Persistir estado en PostgreSQL.

Ejemplo de endpoints del Consumer Identity Hub:

Endpoint	Puerto interno	Puerto host
API base	7080	7280
Identity API	7081	7281
Credentials API	7082	7282
DID web	7083	7283
STS	7084	7284

Los Providers usan puertos internos 8180-8184 y se publican en rangos host diferentes.

5.6 Issuer Service

El Issuer Service actúa como autoridad emisora de credenciales verificables. Emite:

- MembershipCredential para Consumer y Providers.
- TransportCompanyCredential para el Consumer.

Su DID es:

did:web:issuer-service%3A10016:issuer

Expone varios puertos:

API	Puerto
API base	10010
STS	10011
Issuance	10012
Admin	10013
Identity	10015
DID	10016
Status list	9999

El Issuer Service usa PostgreSQL (issuer-postgres) y Vault. Los Control Planes registran al Issuer como trusted issuer para aceptar credenciales emitidas por él.

5.7 Keycloak

Keycloak proporciona el realm local:

logistics-dataspace

Se arranca con import automático desde:

infra/keycloak/import/realm-export.json

Contiene clients, scopes, roles y secretos necesarios para Identity Hubs e Issuer Service. Su base de datos es keycloak-postgres.

5.8 Vault

Vault se usa como almacén de secretos y claves:

- Claves privadas de participantes.
- Secretos STS.
- Claves del Transfer Proxy (private-key, public-key).
- Claves auxiliares del Issuer Service.

La demo usa Vault en modo desarrollo con token:

root

Esto simplifica la demo pero no es una configuración productiva.

5.9 Regulatory Clearance Mock API

El servicio mock-api simula los sistemas backend regulatorios. Está implementado con FastAPI y expone:

- GET /health
- GET /containers/{container_id}/clearance
- GET /containers/{container_id}/customs-clearance

- GET /containers/{container_id}/health-inspection
- GET /containers/{container_id}/civilguard-clearance
- GET /transport-orders/{company_id}/{container_id}/validate

También valida el formato del contenedor:

4 letras + 7 dígitos

Ejemplo:

MSCU7654321

Este backend tiene dos responsabilidades arquitectónicas:

1. Proporcionar los datos que los Providers publican como Assets.
2. Validar órdenes de transporte activas durante la evaluación de políticas.

5.10 API orquestadora local

La API orquestadora vive en `orchestrator_api/` y se ejecuta en el host Windows, fuera de Docker. Su objetivo es permitir que la UI Streamlit contenedorizada lance operaciones del host sin montar el Docker socket ni ejecutar PowerShell desde el contenedor.

Expone:

- GET /health
- GET /commands
- POST /commands/{command_name}/run
- GET /runs
- GET /runs/{run_id}
- GET /runs/{run_id}/log
- POST /runs/{run_id}/stop

Solo acepta comandos de una lista blanca:

Comando	Script
service_start	scripts/service-start.ps1
service_stop	scripts/service-stop.ps1
edc_start	scripts/edc-start.ps1
demo_full	start-edc-and-smoke-three-providers.ps1
smoke_only	smoke-test-three-providers.ps1

Cada ejecución genera metadatos y log en:

`resources/generated/orchestrator-runs/`

Si `ORCHESTRATOR_TOKEN` está definido, todos los endpoints salvo `/health` requieren el header:

X-Orchestrator-Token: <token>

6. Modelo de participantes

El dataspace está formado por cinco identidades principales:

Participante	Rol arquitectonico	DID
Consumer	Empresa transportista consumidora de datos	did:web:consumer-identityhub%3A7083:consumer
Customs	Provider de autorizacion aduanera	did:web:provider-identityhub%3A8183:provider
Health	Provider de inspeccion sanitaria	did:web:health-identityhub%3A8183:health
CivilGuard	Provider de autorizacion Guardia Civil	did:web:civilguard-identityhub%3A8183:civilguard
Issuer	Emisor de credenciales verificables	did:web:issuer-service%3A10016:issuer

La relación entre ellos no es de confianza directa simple. Se apoya en:

- DIDs did:web.
- Credenciales verificables.
- Trusted issuers configurados.
- Scopes DCP.
- Políticas EDC/ODRL evaluadas en cada Provider.

7. Modelo de datos

7.1 Assets

Cada Provider publica un Asset que representa una respuesta regulatoria concreto para un contenedor.

Ejemplo simplificado de Asset:

```
{
  "@id": "asset-clearance-mscu7654321",
  "properties": {
    "name": "Container Clearance Status - MSCU7654321",
    "contenttype": "application/json",
    "useCase": "Port Regulatory Clearance",
    "containerId": "MSCU7654321",
    "dataCategory": "ClearanceStatus"
```

```

},
"dataAddress": {
  "type": "HttpData",
  "baseUrl": "http://regulatory-clearance-api:8081/containers/MSCU7654321/customs-clearance",
  "proxyPath": "true",
  "proxyQueryParams": "true",
  "proxyBody": "true",
  "proxyMethod": "true"
}
}

```

El campo más importante para la autorización dinámica es:

```
containerId
```

La policy puede usar `${containerId}` y la extensión Java lo resuelve contra las propiedades del Asset.

7.2 Policies

Hay una policy permisiva:

```
policy-allow-use
```

y una policy restrictiva:

```
policy-transport-company-valid-order
```

La restrictiva exige:

- `TransportCompanyCredential.role == TransportCompany`
- `TransportOrder.activeForContainer == ${containerId}`

La validación de `TransportOrder.activeForContainer` se resuelve dinámicamente en la extensión Java, llamando al Mock API.

7.3 Contract Definitions

Las Contract Definitions unen:

- Un Asset.
- Una Access Policy.
- Una Contract Policy.

La Access Policy controla si la oferta aparece en el catálogo. La Contract Policy controla si la negociación y transferencia son aceptadas. En el flujo recomendado, la Access Policy suele ser `policy-allow-use` para publicar la oferta y la Contract Policy contiene las restricciones fuertes.

7.4 Artefactos generados

Durante la ejecución se generan artefactos en:

resources/generated/

Entre ellos:

- Catálogos recibidos.
- Solicitudes de negociación.
- Solicitudes de transferencia.
- EDRs.
- Datos descargados por Provider.
- Resultado agregado final.
- Eventos JSONL para la UI.
- Estado de ejecución y logs de scripts.

8. Seguridad y autorización

8.1 Capas de seguridad

La demo combina varias capas:

Capa	Mecanismo
Management APIs	API keys locales (consumer-api-key, provider-api-key).
Identidad federada	DIDs did:web
Autenticacion DCP	STS tokens emitidos por Identity Hub.
Confianza	Trusted issuer registry y tipos soportados.
Credenciales	MembershipCredential, TransportCompanyCredential
Políticas	ODRL evaluadas por el Policy Engine de EDC.
Secretos	Vault para claves privadas, públicas y secretos.
Transferencia	EDR con endpoint y token de autorización.
Orquestacion UI	Token local ORCHESTRATOR_TOKEN y lista blanca de comandos.

8.2 MembershipCredential

La función MembershipCredentialEvaluationFunction valida que:

- El operador sea EQ.
- El operando derecho sea active.
- Existe un ParticipantAgent.

- El agente tenga una credencial cuyo tipo termine en MembershipCredential.
- La credencial incluya un claim membership.
- El claim membership.since sea una fecha anterior al instante actual.

Esta credencial se evalúa en:

- Catálogo.
- Negociación contractual.
- Transferencia.

8.3 TransportCompanyCredential

La función TransportCompanyRoleFunction valida que el Consumer tenga una credencial de tipo TransportCompanyCredential con:

```
role = TransportCompany
```

También existe una aceptación específica para el DID del Consumer demo cuando coincide con el rol esperado. Esto ayuda a la demo local, aunque en un entorno productivo convendría evitar excepciones hardcodeadas o hacerlas configurables.

8.4 Orden de transporte activa

La función TransportOrderActiveForContainerFunction implementa una restricción dinámica:

```
TransportOrder.activeForContainer == ${containerId}
```

Su funcionamiento es:

1. Comprueba que el operador sea EQ.
2. Si el operando es \${containerId} y el contexto es catálogo o negociación, puede diferir la validación cuando aún no hay Asset concreto.
3. En transferencia, resuelve el Asset a partir del contractAgreement.assetId.
4. Extrae el containerId de las propiedades del Asset.
5. Llama al endpoint:
`http://regulatory-clearance-api:8081/transport-orders/TC-A/{containerId}/validate`
6. Acepta solo si la respuesta contiene:

```
{
  "transportOrderValid": true
}
```

Este punto es importante: la autorización no depende solo de credenciales estáticas, sino también de un dato operacional externo.

8.5 Scopes DCP

La extensión DcpPatchExtension registra scopes por defecto:

- org.eclipse.dspace.dcp.vc.type:MembershipCredential:read
- org.eclipse.dspace.dcp.vc.type:TransportCompanyCredential:read

Para el Consumer se añade el scope de TransportCompanyCredential en egress. Todos los participantes usan MembershipCredential. Además, DataAccessCredentialScopeExtractor traduce constraints de política a scopes de credenciales, por ejemplo:

- Constraints TransportCompanyCredential.* requieren scope de lectura de TransportCompanyCredential.
- Constraints DataAccess.* requieren scope de lectura de DataProcessorCredential.

9. Flujos principales

9.1 Arranque completo

El flujo recomendado es:

```
docker compose -f .\docker-compose.infra.yml up -d
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\start-edc-and-smoke-three-providers.ps1
```

El script realiza:

1. Arranque de servicios base EDC, Identity Hubs e Issuer Service y Mock API.
2. Espera activa hasta readiness de Identity Hubs e Issuer.
3. Obtención de tokens OAuth2 desde Keycloak.
4. Activación o reprovisionado de participantes.
5. Emisión de MembershipCredential.
6. Emisión de TransportCompanyCredential.
7. Provisionado de claves en Vault.
8. Construcción de la imagen del Control Plane.
9. Arranque de Control Planes y Data Planes.
10. Registro de Assets, Políticas y Contract Definitions en los Providers.
11. Verificación de Data Planes disponibles.
12. Ejecución del smoke test.
13. Agregación de resultados.

9.1.1 Modo servicio local

El modo servicio local encapsula la demo mediante tres compose:

```
docker compose -f .\docker-compose.infra.yml -f .\docker-compose.edc.yml -f .\docker-  
compose.service.yml up -d --build
```

El arranque recomendado es:

```
.\scripts\orchestrator-start.ps1 -Background  
.\scripts\service-start.ps1
```

scripts/service-start.ps1 crea .env si no existe, crea resources/generated, arranca los compose y solo imprime las URLs finales si Docker Compose termina correctamente. Si existe un contenedor externo llamado vault, muestra instrucciones explícitas para liberar ese nombre.

scripts/orchestrator-start.ps1 crea .venv-orchestrator, instala orchestrator_api/requirements.txt y arranca Uvicorn. Si el puerto 8765 ya está ocupado, consulta /health: si responde, informa de que el orquestador ya está activo; si no responde, indica cómo identificar y parar el PID.

9.2 Agregación de resultados

El resultado agregado esperado para la demo es:

```
{  
  "containerId": "MSCU7654321",  
  "customsStatus": "CLEARED",  
  "healthInspectionStatus": "CLEARED",  
  "civilGuardStatus": "CLEARED",  
  "overallStatus": "READY_FOR_PICKUP",  
  "blockingAuthorities": []  
}
```

Arquitectónicamente, la agregación vive en la capa de automatización/demo, no dentro de EDC. EDC se encarga de descubrir, negociar y transferir datos. La interpretación de negocio de la stres respuestas se realiza después de descargarlas.

10. Arquitectura de la UI Streamlit

La UI se encuentra en ui/ y tiene tres pantallas:

1. Dashboard principal: ui/app.py.
2. Flujo manual por Provider: ui/pages/1_Manual_Provider_Flow.py.
3. Provisioning de Provider: ui/pages/2_Provider_Provisioning.py.

La UI puede ejecutarse de dos formas:

- En local, mediante `python -m streamlit run .\ui\app.py`.

- Como contenedor Docker, mediante docker-compose.service.yml.

Cuando corre en Docker, las llamadas al host se adaptan usando host.docker.internal. Los helpers comunes están en ui/common.py.

10.1 Dashboard principal

Responsabilidades:

- Lanzar scripts PowerShell en segundo plano.
- Evitar ejecuciones solapadas.
- Leer estado de ejecución desde resources/generated/ui-run-state.json.
- Leer eventos desde resources/generated/ui-events.jsonl.
- Mostrar progreso gloabl.
- Mostrar estado por Provider.
- Mostrar últimos mensajes de log.
- Mostrar JSON originales y artefactos generados.

Los botones principales se mapean a comando del orquestador:

Botón	Comando orquestador	Script final
Ejecutar demo completa	demo_full	start-edc-and-smoke-three-providers.ps1
Arrancar EDC sin smoke	edc_start	scripts/edc-start.ps1
Ejecutar solo smoke test	smoke_only	smoke-test-three-providers.ps1

Al lanzar una acción desde la UI se reinician los artefactos monitorizados y la pantalla se refresca cada segundo mientras existe un run RUNNING en el orquestador o eventos recientes en ejecución. La UI muestra la tabla de runs, pero no muestra el log completo del run para evitar ruido visual.

La UI monitoriza pasos globales como:

- infra_check
- identityhubs_ready
- issuer_ready
- participants_activation
- membership_credentials
- transport_company_credential
- vault_provisioning
- controlplanes_dataplanes
- assets_policies_contracts
- dataplanes_available
- smoke_test

- aggregation
- result

Y pasos por Provider:

- catalog
- contract_negotiation
- transfer
- edr
- download

10.2 Flujo manual por Provider

Esta pantalla permite ejecutar manualmente el flujo EDC contra un Provider concreto:

1. Seleccionar Provider.
2. Solicitar catálogo al Consumer Management API.
3. Extraer ofertas del catálogo.
4. Enriquecer ofertas consultando Contract Definitions del Provider.
5. Seleccionar oferta.
6. Negociar contrato.
7. Esperar estado FINALIZED.
8. Iniciar transferencia.
9. Esperar estado STARTED o COMPLETED.
10. Obtener EDR.
11. Descargar dato desde el Data Plane.

La UI normaliza endpoints internos Docker, localhost y host.docker.internal para funcionar tanto desde el host como desde el contenedor. Por ejemplo:

`http://provider-dataplane:19294 -> http://localhost:19294`

Dentro de Docker, si recibe localhost, lo convierte a host.docker.internal; si recibe nombres de servicios Compose, los prueba directamente y después aplica fallback al host cuando corresponde.

10.3 Provisioning de Provider

Esta pantalla actúa como consola visual para Management API de cada Provider. Permite:

- Crear, consultar, actualizar y borrar Assets.
- Crear, consultar, actualizar y borrar Policies.
- Crear, con sultar, actualizar y borrar Contract Definitions.

- Validar que el containerId del Asset coincide con el incluido en el backend endpoint.
- Validar que el containerId del Asset coincide con el de la Contract Policy.
- Listar Assets, Policies y Contract Definitions disponibles.
- Validar backend endpoints desde el host.

Una decisión relevante: para Assets y Contact Definitions usa PUT en actualizaciones, evitando borrar recursos referenciados. Para Policies, como EDC no expone PUT, usa DELETE+POST y recrea Contract Definitions asociadas.

11. Despliegue local

11.1 Infraestructura base

Docker-compose.infra.yml levanta:

- keycloak-postgres
- keycloak
- vault

Comando:

```
docker compose -f .\docker-compose.infra.yml up -d
```

11.2 Servicios EDC

Docker-compose.edc.yml levanta:

- 4 Control Planes.
- 4 Data Planes.
- 4 Identity Hubs.
- 4 PostgreSQL para Identity Hubs.
- Issuer Service.
- PostgreSQL del Issuer.
- Mock API regulatorio.

Comando conjunto:

```
docker compose -f .\docker-compose.infra.yml -f .\docker-compose.edc.yml up -d
```

11.3 Modo servicio UI

docker-compose.service.yml añade el servicio:

- puerto-dataspace-ui, construido desde ui/Dockerfile

La UI monta resources/generated para leer y escribir eventos y artefactos, y usa variables como:

- `RUNNING_IN_DOCKER=true`
- `ORCHESTRATOR_URL=http://host.docker.internal:8765`
- `ORCHESTRATOR_TOKEN=dev-orchestrator-token`

No se despliega la API orquestadora dentro de Docker; se ejecuta en el host Windows.

11.4 Puertos principales

Participante	Identity Hub host	Management API	DSP	Control API	Data Plane publico
Customs	7180-7184	19193	19292	19194	19294
Consumer	7280-7284	29193	29292	29194	29294
Health	7380-7384	21193	21292	21194	21294
CivilGuard	7480-7484	22193	22292	22194	22294

Servicios comunes:

Servicio	Puerto
Keycloak	8080
Vault	8200
Mock API	8081
Issuer Service API	10010
Issuer DID	10016
Issuer status list	9999
UI Streamlit	8501
Orchestrator API	8765

12. Build y empaquetado

El proyecto Gradle incluye los módulos:

- `:extensions:example-extension`
- `:extensions:dcp-impl`
- `:runtimes:controlplane`
- `:runtimes:dataplane`

El Control Plane genera:

```
runtimes/controlplane/build/libs/controlplane.jar
puerto-edc-controlplane:latest
```

El Data Plane genera:

```
runtimes/dataplane/build/libs/dataplane.jar
dataplane:latest
puerto-edc-dataplane:latest
```

El README indica que para la demo se requiere la imagen local:

```
puerto-edc-dataplane:latest
```

Por tanto, conviene mantener alineado el nombre de imagen producido por el build del Data Plane con el nombre usado en Docker Compose.

13. Tests

La extensión extensions/dcp-impl incluye tests unitarios para:

- Extracción de scopes DCP.
- Mapeo de scopes requeridos.
- Evaluación de MembershipCredential.
- Evaluación de TransportCompanyCredential.role.
- Evaluación de TransportOrder.activeForContainer.
- Rechazo de operadores no soportados.
- Rechazo de credenciales ausentes o malformadas.
- Rechazo de fechas futuras o claims incorrectos.

Comando:

```
.\gradlew.bat test
```

Estos tests validan la lógica crítica de autorización si necesidad de levantar Docker.

14. Decisiones arquitectónicas destacables

Separación Control Plane / Data Plane

La arquitectura sigue el modelo EDC recomendado: el Control Plane negocia y autoriza; el Data Plane transfiere. Esto reduce el acoplamiento entre gobierno de datos y movimiento de datos.

Un Identity Hub por participante

Cada participante tiene su propia identidad, base de datos y endpoints DID. Esto representa mejor un dataspace federado que una identidad centralizada única.

Credenciales verificables como base de autorización

La autorización no depende solo de API keys. Las políticas se basan en claims verificables presentados vía DCP.

Validación operacional externa

La existencia de una orden activa se consulta en runtime contra el Mock API. Esto permite que la decisión de acceso dependa de estado de negocio actual.

UI como capa de observabilidad y operación local

Streamlit no forma parte del core EDC, pero reduce la complejidad operativa de la demo. Sirve como panel de control, inspector de artefactos y consola de provisioning.

Orquestador local con lista blanca

La API orquestadora no acepta comandos arbitrarios: solo nombres de comandos predefinidos. Esta decisión mantiene el modo demo operable desde Docker sin dar al contenedor UI acceso directo al Docker socket ni a PowerShell.

Artefactos JSON versionables

Assets, Policies, Contract Definitions, catalog requests y transfer requests viven como JSON en resources/. Esto hace que la demo sea auditable y reproducible.

15. Riesgos, limitaciones y puntos de mejora

Seguridad de la demo

La demo usa:

- Vault dev token root.
- API keys hardcodeadas.
- did:web con HTTPS desactivado
- Secrets de demostración.
- Puertos expuestos localmente.

Esto es adecuado para prototipo, pero no para producción

Hardcoding de participantes

Algunas funciones contienen DID's, company id TC-A, endpoints internos y scopes definidos de forma estática. Para evolucionar hacia una arquitectura más reusable, deberían moverse a configuración.

Persistencia de Control Plane y Data Plane

Las dependencias SQL de Control Plane y Data Plane aparecen comentadas. Si se requiere durabilidad real de negociaciones, transferencias y catálogos, convendría activar persistencia SQL.

16. Conclusiones

La aplicación implementa una demo bastante completa de dataspace portuario. Su valor arquitectónico está en que no simula el intercambio federado con llamadas directas, sino que utiliza los bloques principales de EDC:

- Catálogo DSP.

- Políticas ODRL.
- Negociación contractual.
- Transferencias HttpData-PULL.
- Data Planes separados.
- DCP con credenciales verificables.
- Identity Hubs por participante.
- Issuer Service como autoridad de credenciales.

El proyecto combina una arquitectura federada con herramientas de demo local. Los scripts hacen reproducible el escenario completo y la UI Streamlit facilita explorarlo, diagnosticarlo y provisionarlo sin memorizar todas las llamadas Management API.